vector, array, string, and graph storage

C++ blocks are independent main-body fragments. Add the named standard headers, then use: using namespace std; int main() { /* fragment */ }. Complete harnesses are in the code archive. n means container size; ranges are [first,last).

vector: resizable, contiguous storage

Headers: <vector>, <algorithm>

```
vector<int> v{3, 1, 3};
v.reserve(10);               // size still 3
v.push_back(7); v.emplace_back(5);
v[1] = 2;
auto it = v.insert(v.begin() + 1, 9);
it = v.erase(it);            // next iterator
v.pop_back();
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
for (int& x : v) ++x;        // changes elements
// v is now {3, 4, 8}
```

Operation	Return / cost
vector<int> v(n, 0)	n zeros; O(n)
v[i] / v.at(i)	Reference; O(1). at checks bounds.
push_back / emplace_back	Append; amortized O(1), worst O(n).
pop_back()	void; O(1); requires nonempty.
insert(pos, x)	Iterator to new item; O(n) worst.
erase(pos), erase(l, r)	Iterator after erased range; O(n).
size() / empty()	Count / bool; O(1).
clear() / resize(n)	Clear O(size); resize may be linear.
reserve(n)	Capacity only; O(size) if relocating.

WATCH: reserve does not create elements; v[0] is invalid when size()==0. begin()+i must stay in range; end() is one past the last element and cannot be dereferenced.

Invalidation: reallocation invalidates every iterator, pointer, and reference. Without reallocation, insertion invalidates at/after its position; erasure invalidates at/after the erased position. Reacquire end() after size changes. clear() keeps capacity.

Graph containers: select by operation

Headers: <vector>, <utility>, <tuple>

```
int n = 4;
vector<vector<int>> g(n);
g[0].push_back(2);           // one directed edge
vector<vector<pair<int,int>>> wg(n);
wg[0].push_back({2, 7});     // neighbor, weight
vector<tuple<int,int,int>> edges{{0, 2, 7}};
```

Adjacency lists: O(V+E) storage. An undirected edge needs both directions; an edge list may store it once. Full input examples: p. 6; traversal: p. 23.

FLAGS: vector<bool> is a packed specialization with proxy element references. vector<char> visited(n,0) is often simpler when ordinary element-reference behavior matters. Do not assume vector<bool> behaves like an array of bool objects.

array: fixed size, no insertion or erasure

Headers: <array>, <algorithm>

```
array<int, 4> a{};           // all zero
a.fill(5); a[0] = 2;
sort(a.begin(), a.end());
int lo = a.front(), hi = a.back();
for (const int x : a) { (void)x; }
// size()==4; empty()==false
```

Header <array>. N is part of array<T,N> and must be compile-time constant. Index/at/front/back: O(1); fill: O(N). N=0 is legal, but front/back and element access are not. array<int,4> a; leaves integer elements uninitialized.

string: search, edit, and inspect characters

Headers: <string>, <algorithm>, <cctype>

```
string s = "kabul";
s[0] = 'K'; s += " team";
auto pos = s.find("team");
string part;
if (pos != string::npos) part = s.substr(pos, 4);
s.replace(6, 4, "ICPC");
s.insert(5, ":"); s.erase(5, 1);
for (char& ch : s)
    ch = static_cast<char>(
        tolower(static_cast<unsigned char>(ch)));
// s == "kabul icpc"; part == "team"
```

Operation	Meaning / cost
size(), length(), empty()	Count / bool; O(1).
s[i], s.at(i)	char reference; at checks; O(1).
push_back(c), pop_back()	Add/remove final char; pop needs data.
find(t), find(t, start)	First match position or string::npos.
substr(pos, len)	Copy at most len chars; O(copied length).
insert / erase / replace	Positions and counts; can move O(n) chars.
compare(t)	Negative / zero / positive; lexical order.
sort(begin,end), reverse(...)	Permute characters; O(n log n) / O(n).

WATCH: substr takes a LENGTH, not an end index; substr(pos) takes the suffix. Use auto/size_t for find results and test npos. A char uses single quotes, a string double quotes. Empty-pattern find succeeds at allowed positions.

String search is not guaranteed constant time. Use O(n*m) as a conservative planning bound for naive substring matching; KMP appears on p. 41. Repeated middle edits or repeated large copies can become quadratic. Reacquire string iterators after mutation.

Character classification: pass unsigned-char values to tolower/isdigit/etc., except EOF. C++ string indexes bytes, not Unicode characters. These contest examples assume ASCII. Prefer string over vector<char> when substring operations matter.

Pairs, tuples, and key-value containers

Choose operations by their side effects: a lookup can accidentally insert. map orders keys; unordered_map does not. Costs assume n stored entries and constant-time key operations unless noted.

map: ordered keys and precise access semantics

C++17 expression	Behavior
mp.find(k)	Iterator or end(); no insertion.
mp.count(k)	0 or 1 for map; no insertion.
mp[k]	Reference; inserts default value if absent.
mp.at(k)	Reference; absent key throws out_of_range.
mp.erase(k)	Count erased: 0 or 1.
mp.erase(it)	Iterator to next element; it must be valid.
mp.emplace(k,v)	pair<iterator,bool>; true only if inserted. Existing value retained.
mp.insert_or_assign(k,v)	Insert or overwrite; iterator/bool; bool true only for a new key.
mp.contains(k)	C++20: DO NOT use in C++17.

Headers: <map>

```
map<int, int> mp{{2, 5}};
bool seen = mp.find(3) != mp.end();
mp[3] += 4;                  // inserts 0, then adds
auto [it, inserted] = mp.emplace(2, 99);
mp.at(2) = 6;
auto removed = mp.erase(42);
// seen=false; inserted=false; removed=0
```

Use find for read-only presence checks. if (mp[k]) inserts on a miss and also confuses a stored zero with absence. Map keys are const; update the mapped value, or erase/reinsert to change a key.

Safe iteration and deletion

Headers: <map>

```
map<int, int> mp{{1, 0}, {2, 5}, {3, 0}};
for (auto it = mp.begin(); it != mp.end(); ) {
    if (it->second == 0) it = mp.erase(it);
    else ++it;
}
for (auto& [key, value] : mp) ++value;
// mp now contains only {2, 6}
```

Insertion into map keeps existing iterators/references valid; erasure invalidates only erased elements. Never increment an erased iterator. Use const auto& [k,v] for read-only iteration; auto [k,v] copies each entry.

Ordered lookup: mp.lower_bound(k) gives first key >=k; upper_bound(k) first key >k. Both O(log n). begin()->first is minimum, rbegin()->first maximum; guard empty(). size()/empty() are O(1); clear() is O(n).

pair and tuple: records, keys, and unpacking

Headers: <utility>, <tuple>, <string>

```
pair<int, string> p{2, "Ali"};
auto [score, name] = p;     // copies
auto& [refScore, refName] = p;
refScore = 4;              // modifies p.first
tuple<int, int, string> t{2, 7, "K"};
get<1>(t) = 9;
int a, b; string label;
tie(a, b, label) = t;
// score==2; p.first==4; b==9
```

pair uses .first/.second; tuple uses get<compile_time_index>(). Both compare lexicographically, component by component. make_pair/make_tuple infer types. tie(a,b) compares existing variables without copying; tie(a,ignore) skips an assignment field. Member access is O(1); comparisons may inspect strings.

unordered_map: exact lookup without key order

Headers: <unordered_map>, <string>

```
unordered_map<string, int> freq;
freq.reserve(100);          // plan capacity first
++freq["a"]; ++freq["a"];
auto it = freq.find("b");
bool missing = (it == freq.end());
freq.insert_or_assign("a", 7);
freq.erase("b");
// one entry: a -> 7; iteration order unspecified
```

Operation	map	unordered_map
Find/count/at	O(log n)	Avg O(1), worst O(n)
Insert/operator[]	O(log n)	Avg O(1), worst O(n)
Erase by key	O(log n)	Avg O(1), worst O(n)
Ordered traversal	Yes	No
lower/upper_bound	O(log n)	Unavailable
Size/empty	O(1)	O(1)

Complexities above assume constant-time key comparison/hash. Hashing a length-L string costs O(L); ordered string comparisons may also inspect O(L) characters. Adversarial collisions can make a sequence of hash-table operations quadratic.

Hash-table insertion may rehash and invalidate all iterators, but not references/pointers to existing elements. erase invalidates the erased element only. reserve/rehash can invalidate iterators; reserve before traversing. The safe erase loop at left also works for unordered_map if you do not insert during it.

CHOICE: use map for ordered keys, predecessor/successor queries, or predictable logarithmic lookup. Use unordered_map for exact-key counting when average-case speed is appropriate. Plain arrays are simpler for a small known alphabet (LTRFRQ/SUGITEMS).

PAIR KEYS: map<pair<int,int>,T> uses lexical ordering. C++17 does not provide a default std::hash for pair/tuple; unordered_map with those keys needs a custom hasher. Do not assume all comparable types are hashable.

Sets, stacks, queues, and deques

Use the access pattern to choose a container: ordered neighbors -> set; membership -> hash set; newest -> stack; oldest -> queue; both ends -> deque. C++ blocks use the fragment convention on p. 7.

set: unique sorted values

Headers: <set>, <iterator>

```
set<int> s{4, 1, 4, 7};    // {1,4,7}
auto [it, added] = s.insert(3);
bool has = s.count(4) != 0;
auto p = s.lower_bound(4);
if (p != s.end()) s.erase(p);
int lo = *s.begin(), hi = *s.rbegin();
auto k = s.lower_bound(3);
bool hasPrev = (k != s.begin());
int previous = hasPrev ? *prev(k) : -1;
// {1,3,7}; previous==1
```

Operation	Return / cost
insert(x)	iterator,bool; O(log n)
find(x), count(x)	Iterator/end or 0/1; O(log n)
erase(x)	0/1; O(log n)
erase(it)	Next iterator; amortized O(1)
lower_bound / upper_bound	First >= / >; O(log n)
begin(), rbegin()	Smallest/largest if nonempty
size(), empty(), clear()	O(1), O(1), O(n)

Values are keys: you cannot assign through a set iterator. Change a value by erase + insert. Do not decrement begin(), dereference end(), or use *rbegin() on an empty set. Insertion preserves iterators; erasure invalidates only removed elements.

multiset permits duplicates. count(x) and erase(x) may touch all k matches: O(log n+k). erase(x) removes ALL copies; to remove one, find(x), check end(), then erase(it). set and multiset live in <set>.

unordered_set: membership, no sorted endpoints

Headers: <unordered_set>

```
unordered_set<int> seen{3, 3, 8};
seen.reserve(100);
seen.insert(5);
bool found = seen.find(8) != seen.end();
auto erased = seen.erase(3);
// {5,8}, in an unspecified iteration order
```

Find/insert/erase by key: average O(1), worst O(n). size/empty: O(1); iteration/clear: O(n). No minimum, maximum, or lower_bound operation; scan O(n) if needed. Rehash/invalidation rules match unordered_map (p. 8). No contains in C++17.

SET ITERATION: for (int x : s) reads each value. To remove selected values, use it=s.erase(it) or ++it as on p. 8. Neither ordered nor unordered set supports operator[].

stack and queue: restricted access adapters

Headers: <stack>, <queue>

```
stack<int> st;
st.push(4); st.emplace(8);
int last = st.top(); st.pop();
queue<int> q;
q.push(4); q.emplace(8);
int first = q.front(); q.pop();
int newest = q.back();
// last=8; first=4; newest=8
```

Operation	stack	queue
Insert	push/emplace	push/emplace
Inspect	top()	front(), back()
Remove	pop(): top	pop(): front
Order	LIFO	FIFO
Size / empty	size()/empty()	size()/empty()
Complexity	O(1) each	O(1) each

These O(1) costs use the default deque backing container. pop() returns void, not the removed value: read before popping. Check !empty() first. No iterators, sorting, indexing, find, or erase-by-value; choose another container if those operations are needed.

Use stack for undo / DFS / unmatched positions; queue for BFS. To reset an adapter: st = stack<int>{}; q = queue<int>{}; destroying old elements takes linear work. A vector can serve as a stack with back/push_back/pop_back.

deque: both ends + random access

Headers: <deque>, <algorithm>

```
deque<int> d{2, 4};
d.push_front(1); d.push_back(5);
d[1] = 3;
d.pop_front(); d.pop_back();
sort(d.begin(), d.end());
int first = d.front(), last = d.back();
// d == {3,4}
```

Operation	Cost / caveat
Push/pop at either end	O(1); pop requires nonempty
Index/at/front/back	O(1); at checks bounds
Middle insert/erase	O(n) worst case
Linear find / iteration	O(n)
Size/empty/clear	O(1) / O(1) / O(n)

Deque storage is not one contiguous array. After insert/erase, reacquire iterators; end insertion can invalidate iterators even when references to old elements survive. For monotonic queue patterns keep indices, not mutable iterator positions.

Heaps, sorting, and comparators

A priority queue repeatedly exposes one extreme; sorting orders an entire range. Comparators are executable ordering rules: specify each tie-break explicitly and keep the relation strict.

priority_queue: largest or smallest first

Headers: <queue>, <vector>, <functional>, <utility>

```
priority_queue<int> mx;
mx.push(3); mx.push(7); mx.push(2);
int biggest = mx.top(); mx.pop();
priority_queue<int, vector<int>, greater<int>> mn;
mn.push(3); mn.push(7); mn.push(2);
int smallest = mn.top(); mn.pop();
using Node = pair<long long, int>;
priority_queue<Node, vector<Node>,
               greater<Node>> pq;
pq.push({9, 2}); pq.push({4, 7});
// pq.top() == {4,7}: distance first, vertex second
```

Operation	Cost / result
top()	O(1); const reference
push / emplace	O(log n) heap work; vector growth amortized
pop()	O(log n); void, read top first
size() / empty()	O(1)
priority_queue<int> h(b,e)	O(n) heap construction from a range

Guard top/pop with !empty(). No direct iteration, decrease-key, or arbitrary erase. For Dijkstra, push a new (distance,vertex) and discard stale entries when popped (p. 25). A vector growth step may make an individual push O(n).

Custom heap: reverse the priority relation

Headers: <queue>, <vector>

```
struct Job { int priority, id; };
struct Cmp {
    bool operator()(const Job& a,
                    const Job& b) const {
        if (a.priority != b.priority)
            return a.priority < b.priority;
        return a.id > b.id;
    }
};
priority_queue<Job, vector<Job>, Cmp> q;
q.push({8, 4}); q.push({8, 2}); q.push({3, 1});
// Highest priority, then smallest ID: {8,2}
```

HEAP RULE: cmp(a,b)==true means a has LOWER priority than b. For sort, true means a goes BEFORE b. The comparator must still be strict and consistent. Do not modify a queued object through external mutable state used by its comparator.

Build from a range; drain a copy

Headers: <vector>, <queue>

```
vector<int> a{3, 1, 4};
priority_queue<int> q(a.begin(), a.end());
auto copy = q; vector<int> order;
while (!copy.empty()) {
    order.push_back(copy.top()); copy.pop();
}
// order={4,3,1}; original q still has 3 items
```

Copy: O(n) time/space; drain: O(n log n). Pop q to consume it. top() is read-only; replace it with pop then push.

sort, stable_sort, reverse

Headers: <vector>, <algorithm>, <functional>, <utility>

```
vector<int> a{4, 1, 3};
sort(a.begin(), a.end());          // 1,3,4
sort(a.begin(), a.end(), greater<int>());
reverse(a.begin(), a.end());       // 1,3,4 again
vector<pair<int,int>> p{{2,3},{1,8},{2,1}};
sort(p.begin(), p.end());          // lexicographic
sort(p.begin(), p.end(), [](const auto& x,
                            const auto& y) {
    if (x.first != y.first)
        return x.first < y.first;
    return x.second > y.second;
});                        // (1,8),(2,3),(2,1)
```

Headers: <vector>, <algorithm>, <string>

```
struct Row { int score; string name; };
vector<Row> r{{5,"Z"},{9,"B"},{5,"A"}};
stable_sort(r.begin(), r.end(),
    [](const Row& a, const Row& b) {
        return a.score > b.score;
    });
// B, Z, A: equal scores keep their input order
```

Algorithm	C++17 guarantee
sort	O(n log n) comparisons; not stable
stable_sort	O(n log n) comparisons if enough extra memory; otherwise O(n log^2 n)
reverse	O(n) swaps; reverses whole range
sort(rbegin,rend)	Descending with default <

Sorting rearranges elements. Save original indices if the output needs original positions. Ranges are [first,last): use begin()+l, begin()+r for [l,r), with valid bounds. Neither sort nor stable_sort returns a sorted container; both return void.

COMPARATOR CHECK: cmp(x,x) must be false. Use < or >, never <= or >=. Break ties only when the statement requests it. Avoid subtraction comparisons such as a-b<0: they can overflow. Comparing floating values with an arbitrary epsilon may violate transitivity.

Sorting pairs uses both fields by default; sorting tuples uses all fields. A custom comparator can select a subset. stable_sort preserves the prior order only for elements equivalent under that comparator; it does not automatically sort ties alphabetically.

CONTEST LINK: LTRFRQ needs frequency descending then letter ascending. BESTFRIEND needs multiple ordered criteria. CPU/laptop Rentals need start-time sorting plus end-time heap logic; sorting by itself does not count simultaneous demand.

Algorithms, bits, and useful headers

Headers are listed with each independent C++ fragment. A returned iterator is not an element value. Never dereference a search result until you know it is not end().

Bounds and membership: sorted ranges

Headers: <vector>, <algorithm>

```
vector<int> a{1, 3, 3, 7};
auto lo = lower_bound(a.begin(), a.end(), 3);
auto hi = upper_bound(a.begin(), a.end(), 3);
auto count3 = hi - lo;            // 2
auto index = lo - a.begin();      // 1
bool found = binary_search(a.begin(), a.end(), 4);
// found=false; hi points to 7
```

Function	Result
lower_bound(b,e,x)	First element not less than x
upper_bound(b,e,x)	First element greater than x
binary_search(b,e,x)	bool membership
equal_range(b,e,x)	pair {lower_bound,upper_bound}
No boundary exists	Returned iterator is e, not -1

For sorted vector/array: O(log n) comparisons and jumps. Generic bounds on non-random-access iterators can make O(n) increments; use set/map member lower_bound instead. Searches must use the same ordering as sorting, including a custom comparator.

Linear tools: values versus iterators

Headers: <vector>, <algorithm>, <numeric>

```
vector<int> a{3, 1, 3, 2};
long long sum = accumulate(
    a.begin(), a.end(), 0LL);
auto cnt = count(a.begin(), a.end(), 3);
auto hit = find(a.begin(), a.end(), 2);
auto small = min_element(a.begin(), a.end());
auto large = max_element(a.begin(), a.end());
int minValue = *small, maxValue = *large;
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
// a == {1,2,3}; old iterators must not be reused
```

find/min_element/max_element return iterators, not values. Empty range: they return end(); do not dereference. count returns a count. Each is O(n). accumulate uses its initial value type: 0 may overflow as int even if assigned afterward to long long.

unique is O(n) and removes only ADJACENT repeats logically; it returns a new logical end without resizing. Sort first only when losing order is allowed. Erase the tail to resize. To delete every x: a.erase(remove(a.begin(),a.end(),x),a.end()); <algorithm>.

Numeric helpers: exact call syntax

Headers: <vector>, <numeric>, <limits>

```
vector<int> id(4);
iota(id.begin(), id.end(), 0);  // 0,1,2,3
long long g = gcd(12LL, 18LL);  // 6
long long l = lcm(12LL, 18LL);  // 36
auto hi = numeric_limits<long long>::max();
auto low = numeric_limits<double>::lowest();
```

For floating types, min() is the smallest positive normal value; lowest() is the most negative finite value. lcm can overflow even when each input fits. Type/range safeguards continue on pp. 13 and 36.

next_permutation: lexicographic enumeration

Headers: <vector>, <algorithm>

```
vector<int> a{2, 1, 1};
sort(a.begin(), a.end());
int ways = 0;
do { ++ways; } while (
    next_permutation(a.begin(), a.end()));
// ways==3: 112,121,211; a reset to 112
```

Returns true if advanced; false at the last ordering and resets to the first. Start sorted to enumerate every distinct arrangement even with duplicates. O(n) per call, up to n! arrangements. The do-loop visits an empty sequence once.

bitset and unsigned bit operations

Headers: <bitset>, <string>, <cstdint>

```
bitset<8> b(string("00101100"));
b.set(0); b.reset(2); b.flip(7);
bool on = b.test(3); auto ones = b.count();
uint64_t x = 0b101100;
int k = 1; x |= uint64_t{1} << k; // set
x &= ~(uint64_t{1} << k);         // clear
x ^= uint64_t{1} << k;            // toggle
bool present = (x >> k) & 1;
uint64_t lowbit = x & (~x + 1);  // unsigned
```

Operation	Meaning / guard
& | ^ ~	AND / OR / XOR / complement
x & (x-1)	Clear lowest set bit; unsigned x
x && !(x&(x-1))	Positive power of two, unsigned x
b.any/none/all/count()	Any / none / all / number set
b[i], b.test(i)	Bit i (rightmost = 0); test checks
b.to_string()	Fixed-width 0/1 string

Require 0<=k<64 for uint64_t shifts; shifting by 64 is invalid. Prefer unsigned masks; 1<<k starts with signed int. bitset<N> has compile-time width; bulk bit operations scale with its storage (often N/word_bits). No std::popcount or std::bit_width in C++17.

Useful standard headers

Header	Names to recall
<numeric>	accumulate, iota, gcd, lcm
<limits>	numeric_limits<T>::max() and ::lowest()
<cmath>	sqrt, hypot, floor, ceil, round
<iomanip>	fixed, setprecision, setw, setfill
<cstdint> / <climits>	uint64_t / LLONG_MAX
<utility> / <tuple>	pair, swap / tuple, tie, get
<functional> / <iterator>	greater / next, prev, distance

gcd/lcm are C++17; a representable result and valid operand magnitudes are required. iota(b,e,start) fills a sequence in O(n). <bits/stdc++.h> is a non-standard compiler convenience, not required by this manual.

Python containers and library operations

These five Python 3 snippets are independently executable; imports are included. Prefer standard APIs available in established Python 3 versions. Algorithms elsewhere remain primarily C++17.

Lists, tuples, strings

Python 3 - independent snippet

```
a = [3, 1, 3]
a.append(5); a.extend([2])
a.insert(1, 9); a[0] = 4
last = a.pop(); a.remove(3)  # first matching 3
del a[1]
b = sorted(a); a.sort(reverse=True)
piece = a[1:]; copied = a[:]
t = (7, "K"); score, name = t
s = "Kabul team"
words = s.split(); joined = "-".join(words)
pos = s.find("team")        # -1 if absent
```

Lists mutate; strings/tuples do not. append/extend/insert/sort/reverse return None. pop returns the removed value; remove/index raise ValueError if absent. Use x in a, len(a), not a, min(a), max(a); min/max need nonempty data unless a default is provided. A one-item tuple is (x,).

Sets and dictionaries

Python 3 - independent snippet

```
s = {3, 1, 3}; s.add(7); s.discard(9)
other = {1, 8}
both = s & other; either = s | other
only = s - other; xor = s ^ other
d = {"a": 2}; d["b"] = 3
value = d.get("missing", 0)  # no insertion
d["a"] += 1
old = d.pop("b", None)
for key in list(d):          # snapshot for erasure
    if d[key] == 0: del d[key]
pairs = list(d.items())
```

{} is an empty dict; set() is an empty set. dict membership tests KEYS. d[k] raises KeyError on a miss; get does not insert. set.remove raises on a miss, discard does not. Keys/set items must be hashable: use tuples, not lists. Do not change dict/set size during live iteration.

Operation	Typical CPython cost
List indexing / len	O(1)
Append / end pop	Amortized O(1)
Front/middle insert/pop	O(n)
List membership / count	O(n)
Slice / shallow copy	O(k) copied items
List sort / sorted	O(n log n) worst case
dict/set single-key ops	Avg O(1); worst O(n)
Dict/set min/max	O(n); unordered by value

Costs assume constant-time hashing/comparison; large integers/strings add work. a=b aliases a list; a[:] is a shallow copy. Nested mutable rows still need separate copies. Set order is not sorted; use sorted(s). Dict order is insertion order in Python 3.7+, not key order.

Counter, defaultdict, deque

Python 3 - independent snippet

```
from collections import Counter, defaultdict, deque
c = Counter("abaca"); missing = c["z"]
c.update("bb"); c["a"] -= 3
del c["a"]          # zero is not auto-deleted
g = defaultdict(list)
g[0].append(2)              # creates missing list
q = deque([2]); q.appendleft(1); q.append(3)
front = q.popleft(); back = q.pop()
```

Counter reads absent counts as 0; most_common(k) ranks counts. defaultdict[k] inserts, get(k) does not. deque supports O(1) end pushes/pops; middle indexing/search is O(n). Use deque for BFS, not list.pop(0). Check nonempty before popping.

heapq and bisect

Python 3 - independent snippet

```
import heapq
from bisect import (bisect_left, bisect_right,
                    insort)
h = [5, 2, 7]; heapq.heapify(h)
heapq.heappush(h, 1); best = heapq.heappop(h)
mx = [-x for x in [5, 2, 7]]
heapq.heapify(mx); biggest = -heapq.heappop(mx)
a = [1, 3, 3, 7]
l = bisect_left(a, 3); r = bisect_right(a, 3)
found = l < len(a) and a[l] == 3
insort(a, 2)
```

heapify O(n); push/pop O(log n); peek h[0] O(1). Empty pop raises IndexError. Negation makes a portable max-heap. Tuple priorities break ties lexically: add a serial number before an incomparable payload. bisect is O(log n) on sorted lists; insort is O(n) because insertion shifts items.

Sorting, iteration, and itertools

Python 3 - independent snippet

```
from itertools import combinations, permutations
from itertools import product, accumulate, chain
rows = [(2,"B"),(1,"C"),(2,"A")]
ranked = sorted(rows, key=lambda x: (-x[0], x[1]))
ids = list(enumerate(["a","b"], start=1))
pairs = list(zip([1,2], ["a"]))
comb = list(combinations([1,2,3], 2))
perm = list(permutations([1,2]))
bits = list(product([0,1], repeat=2))
pref = [0] + list(accumulate([2,3]))
flat = list(chain.from_iterable([[1,2],[3]]))
```

sorted returns a new list and is stable; key is computed once per item. zip stops at the shortest iterable. itertools returns lazy iterators; list(...) materializes every result. Counts can be n!, C(n,k), or 2^n; laziness does not reduce total enumeration time. groupby groups adjacent equal keys only.